

Implementing Robust State Management with Error Handling

Objective

Real-world AI systems face constant challenges: API timeouts, unexpected user inputs, service outages, and edge cases that never appeared in testing. Building robust state management isn't just about storing data—it's about creating workflows that gracefully handle failures and continue providing value even when things go wrong.

Scenario

You're building a document processing system for a legal firm that handles contract analysis. The system needs to extract key information from contracts, validate legal clauses, and generate summaries. However, the system must be resilient: external legal databases sometimes go offline, OCR services occasionally fail, and some documents have unusual formats that break standard processing.

Your firm processes hundreds of contracts daily, and any system downtime costs thousands of dollars in delayed deals. The system must handle failures gracefully, provide useful results even when some components fail, and maintain detailed logs for compliance auditing.

Design and implement a resilient LangGraph workflow that processes legal documents with comprehensive error handling, fallback mechanisms, and state management that supports debugging and compliance requirements.

Instructions for learners

Task 1: Design Resilient State Schema (3 minutes)

Create a state structure that supports both business logic and resilience tracking:

Execution tracking (processing steps, timestamps, success/failure status)

Error handling data (fallback usage, retry counts, error logs)

Compliance information (processing audit trail, data sources used)

Task 2: Implement Error Handling Strategies (4 minutes)

Design error handling for three critical failure scenarios

External API Failure: OCR service or legal database unavailable

Data Quality Issues: Corrupted or unreadable document content

Processing Timeout: Complex documents that exceed processing time limits

For each node, define:

- Detection method (how you identify the failure)
- Fallback strategy (alternative processing approach)
- State updates (what information to record about the failure)

Task 3: Create Conditional Routing Logic (3 minutes)

Design routing logic that adapts based on

Document complexity (simple contracts vs. complex multi-party agreements)

System health (current load, service availability)

Processing history (has this document type failed before?)

Urgency level (rush processing vs. standard workflow)

Map out the decision tree that determines which processing path to take based on these factors.

Submission Guidelines

Submit a structured document (PDF, Word, or well-formatted text) containing

- State Schema: Complete state structure with field descriptions
- Error Handling Plan: Three failure scenarios with detection and fallback strategies
- Routing Logic: Decision tree or flowchart showing conditional paths
- Resilience Rationale: Brief explanation of how your design ensures system reliability

Completion Criteria

Comprehensive state schema that supports both business and operational needs

Realistic error handling strategies for each failure scenario

Conditional routing that considers multiple factors

Clear explanation of how the design maintains system reliability

Engage in reflection: Transition to Dialogue

After submitting your resilient workflow design, you'll engage in a structured conversation to evaluate your error handling strategies and explore how they align with production system requirements.